

Arrest_Data_Modeling

Luca Cevaer-Corey

```
#install.packages(c("sf", "tigris", "ggplot2")) # if needed
```

```
library(sf)
```

```
## Warning: package 'sf' was built under R version 4.5.2
```

```
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```
library(tigris)
```

```
## To enable caching of data, set `options(tigris_use_cache = TRUE)`
```

```
## in your R script or .Rprofile.
```

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
options(tigris_use_cache = TRUE)
```

```
# Set project folder
```

```
setwd("~/Desktop/CSE160_Real_Estate/")
```

```
# Read SLD NYC file (the one that has TRACT_GEOID and ZIP)
```

```
sld <- read.csv("sld_ny_city_with_zips.csv", stringsAsFactors = FALSE)
```

```
# Basic structure
```

```
str(sld)[1:10]
```

```
## 'data.frame': 9316 obs. of 121 variables:
```

```
## $ X : int 148837 148838 148838 148839 148839 148840 148841 148841 148842 148842 ...
```

```
## $ OBJECTID : int 148837 148838 148838 148839 148839 148840 148841 148841 148842 148842 ...
```

```
## $ GEOID10 : num 3.61e+11 3.61e+11 3.61e+11 3.61e+11 3.61e+11 ...
```

```
## $ GEOID20 : num 3.61e+11 3.61e+11 3.61e+11 3.61e+11 3.61e+11 ...
```

```
## $ STATEFP : int 36 36 36 36 36 36 36 36 36 36 ...
```

```
## $ COUNTYFP : int 81 81 81 81 81 81 81 81 81 81 ...
```

```
## $ TRACTCE : int 25900 16600 16600 122702 122702 28300 148300 148300 53900 53900 ...
```

```
## $ BLKGRPCE : int 2 2 2 1 1 5 4 4 1 1 ...
```

```
## $ CSA : int 408 408 408 408 408 408 408 408 408 408 ...
```

```
## $ CSA_Name : chr "New York-Newark, NY-NJ-CT-PA" "New York-Newark, NY-NJ-CT-PA" "New York-Newark
```

```
## $ CBSA : int 35620 35620 35620 35620 35620 35620 35620 35620 35620 35620 ...
```

```
## $ CBSA_Name : chr "New York-Newark-Jersey City, NY-NJ-PA" "New York-Newark-Jersey City, NY-NJ-PA
```

```
## $ CBSA_POP : int 19318471 19318471 19318471 19318471 19318471 19318471 19318471 19318471 19318471 19318471
```

```
## $ CBSA_EMP : int 9159786 9159786 9159786 9159786 9159786 9159786 9159786 9159786 9159786 9159786
```

```
## $ CBSA_WRK : int 8946175 8946175 8946175 8946175 8946175 8946175 8946175 8946175 8946175 8946175
```

```
## $ Ac_Total : num 16 22.6 22.6 17.9 17.9 ...
```

```
## $ Ac_Water : num 0 0 0 0 0 0 0 0 0 0 ...
```

```
## $ Ac_Land : num 16 22.6 22.6 17.9 17.9 ...
```

```

## $ Ac_Unpr      : num  16 22.6 22.6 17.9 17.9 ...
## $ TotPop       : int  1278 1568 1568 2363 2363 2614 883 883 909 909 ...
## $ CountHU      : int  448 447 447 962 962 1255 329 329 315 315 ...
## $ HH           : int  405 447 447 962 962 1181 329 329 315 315 ...
## $ P_WrkAge     : num  0.581 0.81 0.81 0.47 0.47 0.753 0.528 0.528 0.754 0.754 ...
## $ AutoOwn0     : int  230 120 120 803 803 634 12 12 132 132 ...
## $ Pct_A00      : num  0.568 0.268 0.268 0.835 0.835 ...
## $ AutoOwn1     : int  155 203 203 129 129 502 123 123 119 119 ...
## $ Pct_A01      : num  0.383 0.454 0.454 0.134 0.134 ...
## $ AutoOwn2p    : int  20 124 124 30 30 45 194 194 64 64 ...
## $ Pct_A02p     : num  0.0494 0.2774 0.2774 0.0312 0.0312 ...
## $ Workers      : int  620 573 573 605 605 1108 244 244 430 430 ...
## $ R_LowWageWk  : int  142 117 117 135 135 241 45 45 78 78 ...
## $ R_MedWageWk  : int  230 220 220 211 211 329 54 54 140 140 ...
## $ R_HiWageWk   : int  248 236 236 259 259 538 145 145 212 212 ...
## $ R_PCTLOWWAGE: num  0.229 0.204 0.204 0.223 0.223 ...
## $ TotEmp       : int  914 142 142 0 0 169 153 153 868 868 ...
## $ E5_Ret       : int  121 54 54 0 0 39 0 0 26 26 ...
## $ E5_Off       : int  8 3 3 0 0 44 4 4 21 21 ...
## $ E5_Ind       : int  331 73 73 0 0 19 57 57 727 727 ...
## $ E5_Svc       : int  396 9 9 0 0 28 92 92 29 29 ...
## $ E5_Ent       : int  58 3 3 0 0 39 0 0 65 65 ...
## $ E8_Ret       : int  121 54 54 0 0 39 0 0 26 26 ...
## $ E8_off       : int  8 3 3 0 0 44 4 4 21 21 ...
## $ E8_Ind       : int  331 73 73 0 0 19 57 57 727 727 ...
## $ E8_Svc       : int  75 7 7 0 0 15 51 51 27 27 ...
## $ E8_Ent       : int  58 3 3 0 0 39 0 0 65 65 ...
## $ E8_Ed        : int  4 1 1 0 0 9 35 35 0 0 ...
## $ E8_Hlth      : int  317 1 1 0 0 4 6 6 2 2 ...
## $ E8_Pub       : int  0 0 0 0 0 0 0 0 0 0 ...
## $ E_LowWageWk  : int  260 38 38 0 0 62 19 19 104 104 ...
## $ E_MedWageWk  : int  246 49 49 0 0 60 45 45 313 313 ...
## $ E_HiWageWk   : int  408 55 55 0 0 47 89 89 451 451 ...
## $ E_PctLowWage: num  0.284 0.268 0.268 0 0 ...
## $ D1A          : num  28.1 19.8 19.8 53.7 53.7 ...
## $ D1B          : num  80 69.4 69.4 131.8 131.8 ...
## $ D1C          : num  57.24 6.29 6.29 0 0 ...
## $ D1C5_RET     : num  7.58 2.39 2.39 0 0 ...
## $ D1C5_OFF     : num  0.501 0.133 0.133 0 0 ...
## $ D1C5_IND     : num  20.73 3.23 3.23 0 0 ...
## $ D1C5_SVC     : num  24.8 0.399 0.399 0 0 ...
## $ D1C5_ENT     : num  3.632 0.133 0.133 0 0 ...
## $ D1C8_RET     : num  7.58 2.39 2.39 0 0 ...
## $ D1C8_OFF     : num  0.501 0.133 0.133 0 0 ...
## $ D1C8_IND     : num  20.73 3.23 3.23 0 0 ...
## $ D1C8_SVC     : num  4.7 0.31 0.31 0 0 ...
## $ D1C8_ENT     : num  3.632 0.133 0.133 0 0 ...
## $ D1C8_ED      : num  0.2505 0.0443 0.0443 0 0 ...
## $ D1C8_HLTH    : num  19.8527 0.0443 0.0443 0 0 ...
## $ D1C8_PUB     : num  0 0 0 0 0 0 0 0 0 0 ...
## $ D1D          : num  85.3 26.1 26.1 53.7 53.7 ...
## $ D1_FLAG      : int  0 0 0 0 0 0 0 0 0 0 ...
## $ D2A_JPHH     : num  2.257 0.318 0.318 0 0 ...
## $ D2B_E5MIX    : num  0.755 0.651 0.651 0 0 ...

```

```
## $ D2B_E5MIXA : num 0.755 0.651 0.651 0 0 ...
## $ D2B_E8MIX : num 0.744 0.561 0.561 0 0 ...
## $ D2B_E8MIXA : num 0.696 0.525 0.525 0 0 ...
## $ D2A_EPHHM : num 0.814 0.449 0.449 0 0 ...
## $ D2C_TRPMX1 : num 0.702 0.465 0.465 0 0 ...
## $ D2C_TRPMX2 : num 0.72 0.463 0.463 0 0 ...
## $ D2C_TRIPEQ : num 0.47 0.164 0.164 0.368 0.368 ...
## $ D2R_JOBPOP : num 0.834 0.166 0.166 0 0 ...
## $ D2R_WRKEMP : num 0.808 0.397 0.397 1 1 ...
## $ D2A_WRKEMP : num 0.678 4.035 4.035 0 0 ...
## $ D2C_WREMLX : num 0.7249 0.0481 0.0481 0 0 ...
## $ D3A : num 25.4 29.5 29.5 15.4 15.4 ...
## $ D3AAO : num 0 0 0 0 0 0 0 0 0 0 ...
## $ D3AMM : num 3.88 13.93 13.93 4.1 4.1 ...
## $ D3APO : num 21.5 15.5 15.5 11.3 11.3 ...
## $ D3B : num 133.6 113.4 113.4 59.5 59.5 ...
## $ D3BAO : num 0 0 0 0 0 0 0 0 0 0 ...
## $ D3BMM3 : num 40.1 0 0 0 0 ...
## $ D3BMM4 : num 80.2 56.7 56.7 0 0 ...
## $ D3BP03 : num 40.1 0 0 35.7 35.7 ...
## $ D3BP04 : num 0 56.7 56.7 35.7 35.7 ...
## $ D4A : num 243 161 161 302 302 ...
## $ D4B025 : num 1 0 0 0 0 ...
## $ D4B050 : num 1 0.13 0.13 0 0 ...
## $ D4C : num 113.3 27.7 27.7 37 37 ...
## $ D4D : num 4542 784 784 1321 1321 ...
## $ D4E : num 0.0887 0.0176 0.0176 0.0157 0.0157 ...
## [list output truncated]
```

```
## NULL
```

```
# Sanity: check that TRACT_GEOID and ZIP exist
stopifnot("TRACT_GEOID" %in% names(sld))
stopifnot("ZIP" %in% names(sld))

# Coerce to character (avoid scientific notation issues)
sld$TRACT_GEOID <- as.character(sld$TRACT_GEOID)
sld$ZIP <- as.character(sld$ZIP)

# Check lengths of GEOIDs
table(nchar(sld$TRACT_GEOID))
```

```
##
## 11
## 9316
```

```
# Peek at a few rows
head(sld[, c("TRACT_GEOID", "countyName", "ZIP")])
```

```
## TRACT_GEOID countyName ZIP
## 1 36081025900 Queens 11377
## 2 36081016600 Queens 11420
## 3 36081016600 Queens 11419
## 4 36081122702 Queens 11365
## 5 36081122702 Queens 11367
## 6 36081028300 Queens 11372
```

```

length(unique(sld$TRACT_GEOID))

## [1] 2164

length(unique(sld$ZIP))

## [1] 280

# Download NY state tracts (sf)
ny_tracts <- tracts(state = "NY", year = 2020, class = "sf")

# GEOID is the 11-digit tract ID
str(ny_tracts$GEOID)

## chr [1:5411] "36047000700" "36047000900" "36047001100" "36047001300" ...

# Optional: restrict to NYC counties via COUNTYFP (Bronx=005, Kings=047, New York=061, Queens=081, Rich
nyc_counties <- c("005", "047", "061", "081", "085")
nyc_tracts <- ny_tracts[nyc_tracts$COUNTYFP %in% nyc_counties, ]

nrow(nyc_tracts) # how many tracts

## [1] 2327

# Use the deduplicated SLD one ZIP per tract
sld_zip <- sld[, c("TRACT_GEOID", "ZIP", "countyName")]
sld_zip$TRACT_GEOID <- as.character(sld_zip$TRACT_GEOID)
sld_zip$ZIP <- as.character(sld_zip$ZIP)

# drop rows with missing ZIP
sld_zip <- sld_zip[!is.na(sld_zip$ZIP) & sld_zip$ZIP != "", ]

# if a tract has multiple ZIPs, keep the first
sld_zip <- sld_zip[!duplicated(sld_zip$TRACT_GEOID), ]

# sanity checks
nrow(sld_zip) # tracts that now have exactly one ZIP

## [1] 2157

length(unique(sld_zip$ZIP))

## [1] 207

# Merge onto NYC tracts
nyc_tracts_merged <- merge(
  nyc_tracts,
  sld_zip[, c("TRACT_GEOID", "ZIP")],
  by.x = "GEOID",
  by.y = "TRACT_GEOID",
  all.x = TRUE
)

# Check ZIP coverage and uniqueness
sum(is.na(nyc_tracts_merged$ZIP)) # tracts with no ZIP

## [1] 301

```

```

length(unique(nyc_tracts_merged$ZIP)) # unique ZIPs

## [1] 204

length(unique(nyc_tracts_merged$GEOID)) == nrow(nyc_tracts_merged) # TRUE = no duplication

## [1] TRUE

# Load cleaned arrests
arrests_clean <- read.csv("NYPD_Arrests_Clean.csv", stringsAsFactors = FALSE)

# Sanity: check coordinates
summary(arrests_clean$Latitude)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  40.50  40.67  40.73  40.74  40.81  40.91

summary(arrests_clean$Longitude)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -74.25 -73.98 -73.92 -73.92 -73.88 -73.70

# Filter to valid coords
arrests_xy <- arrests_clean[
  !is.na(arrests_clean$Latitude) &
  !is.na(arrests_clean$Longitude) &
  !(arrests_clean$Latitude == 0 & arrests_clean$Longitude == 0),
]

nrow(arrests_xy)

## [1] 212055

# Convert to sf points (WGS84)
arrests_sf <- st_as_sf(
  arrests_xy,
  coords = c("Longitude", "Latitude"),
  crs = 4326
)

# Transform both to same CRS (use tracts' CRS)
nyc_tracts_merged <- st_transform(nyc_tracts_merged, 2263) # NY State Plane as example
arrests_sf <- st_transform(arrests_sf, st_crs(nyc_tracts_merged))

# Spatial join: assign each arrest the ZIP from the tract it falls in
arrests_with_zip_sf <- st_join(arrests_sf, nyc_tracts_merged["ZIP"])

# Drop geometry back to data.frame
arrests_with_zip <- as.data.frame(st_drop_geometry(arrests_with_zip_sf))

# How many arrests got a ZIP?
sum(!is.na(arrests_with_zip$ZIP))

## [1] 192046

sum(is.na(arrests_with_zip$ZIP))

## [1] 20009

```

```

# Drop arrests without ZIPs
arrests_with_zip <- arrests_with_zip[!is.na(arrests_with_zip$ZIP), ]

# Clean up weird values
arrests_with_zip$LAW_CAT_CD[arrests_with_zip$LAW_CAT_CD %in% c("", "9", "I")] <- NA

# Total arrests per ZIP
crime_zip <- aggregate(
  ARREST_KEY ~ ZIP,
  data = arrests_with_zip,
  FUN = length
)
names(crime_zip)[names(crime_zip) == "ARREST_KEY"] <- "total_arrests_zip"

# Felony arrests per ZIP
crime_zip_fel <- aggregate(
  I(LAW_CAT_CD == "Felony") ~ ZIP,
  data = arrests_with_zip,
  FUN = function(x) sum(x, na.rm = TRUE)
)
names(crime_zip_fel)[2] <- "felony_zip"

# Misdemeanor arrests per ZIP
crime_zip_mis <- aggregate(
  I(LAW_CAT_CD == "Misdemeanor") ~ ZIP,
  data = arrests_with_zip,
  FUN = function(x) sum(x, na.rm = TRUE)
)
names(crime_zip_mis)[2] <- "misdemeanor_zip"

# Violation arrests per ZIP
crime_zip_viol <- aggregate(
  I(LAW_CAT_CD == "Violation") ~ ZIP,
  data = arrests_with_zip,
  FUN = function(x) sum(x, na.rm = TRUE)
)
names(crime_zip_viol)[2] <- "violation_zip"

# Merge all crime summaries
crime_zip <- merge(crime_zip, crime_zip_fel, by = "ZIP", all = TRUE)
crime_zip <- merge(crime_zip, crime_zip_mis, by = "ZIP", all = TRUE)
crime_zip <- merge(crime_zip, crime_zip_viol, by = "ZIP", all = TRUE)

# Replace any remaining NAs with zero
crime_zip[is.na(crime_zip)] <- 0

head(crime_zip)

##      ZIP total_arrests_zip felony_zip misdemeanor_zip violation_zip
## 1 10001             4303         1552             2693           44
## 2 10002             1960           857             1072            9
## 3 10003             1810           632             1155           22
## 4 10004              43            8              34            0
## 5 10007             688          111             544           30

```

```
## 6 10009          740          276          459          3
```

```
summary(crime_zip$total_arrests_zip)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      2.0   182.0   680.0   955.5  1459.0  5021.0
```

```
summary(crime_zip$felony_zip)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      0.0   69.0   232.0   385.4   605.0  2571.0
```

```
# Read clean 2017 property sales (sf object)
prop_2017_sf <- readRDS("clean_property_2017.rds") # should be sf
```

```
# Sanity: check structure
prop_2017_sf
```

```
## Simple feature collection with 26554 features and 9 fields
## Geometry type: POINT
## Dimension:      XY
## Bounding box:   xmin: 913420 ymin: 121210 xmax: 1066978 ymax: 271768
## Projected CRS: NAD83 / New York Long Island (ftUS)
## First 10 features:
```

##	year	borough	tot_sqft	price	land_sqft	yr_built	bldg_ctgy
## 2	2017	2	1460	305000	1330	1899	01 ONE FAMILY DWELLINGS
## 3	2017	2	1440	178000	1306	1899	01 ONE FAMILY DWELLINGS
## 4	2017	2	1440	449000	1306	1899	01 ONE FAMILY DWELLINGS
## 5	2017	2	1587	140000	1622	1899	01 ONE FAMILY DWELLINGS
## 6	2017	2	1497	246000	1694	1899	01 ONE FAMILY DWELLINGS
## 7	2017	2	1764	420000	3525	1899	01 ONE FAMILY DWELLINGS
## 8	2017	2	1340	380250	3525	1899	01 ONE FAMILY DWELLINGS
## 9	2017	2	2820	325000	1293	1952	01 ONE FAMILY DWELLINGS
## 10	2017	2	2047	127000	2356	1901	01 ONE FAMILY DWELLINGS
## 12	2017	2	2400	350000	2000	1993	02 TWO FAMILY DWELLINGS

##	bldg_num_ctgy	sale_date	geometry
## 2	1	2017-07-18	POINT (1012979 250714)
## 3	1	2017-01-19	POINT (1012988 250709)
## 4	1	2017-07-14	POINT (1012988 250709)
## 5	1	2017-05-12	POINT (1013031 250295)
## 6	1	2017-11-06	POINT (1013008 250243)
## 7	1	2017-02-03	POINT (1012633 249358)
## 8	1	2017-09-29	POINT (1012993 250193)
## 9	1	2017-11-13	POINT (1012903 249615)
## 10	1	2017-08-22	POINT (1012841 248152)
## 12	2	2017-09-05	POINT (1011438 244408)

```
str(prop_2017_sf)
```

```
## Classes 'sf' and 'data.frame': 26554 obs. of 10 variables:
```

```
## $ year      : chr  "2017" "2017" "2017" "2017" ...
## $ borough   : int   2 2 2 2 2 2 2 2 2 2 ...
## $ tot_sqft  : int   1460 1440 1440 1587 1497 1764 1340 2820 2047 2400 ...
## $ price     : num   305000 178000 449000 140000 246000 ...
## $ land_sqft : int   1330 1306 1306 1622 1694 3525 3525 1293 2356 2000 ...
## $ yr_built  : int   1899 1899 1899 1899 1899 1899 1899 1952 1901 1993 ...
## $ bldg_ctgy : chr   "01 ONE FAMILY DWELLINGS" "01 ONE FAMILY DWELLINGS" "01 ONE FAMILY DWELLINGS"
```

```

## $ bldg_num_ctgy: num 1 1 1 1 1 1 1 1 1 2 ...
## $ sale_date : chr "2017-07-18" "2017-01-19" "2017-07-14" "2017-05-12" ...
## $ geometry :sfc_POINT of length 26554; first list element: 'XY' num 1012979 250714
## - attr(*, "sf_column")= chr "geometry"
## - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",...: NA NA NA NA NA NA NA NA NA
## ..- attr(*, "names")= chr [1:9] "year" "borough" "tot_sqft" "price" ...

# Make sure CRS matches tracts
prop_2017_sf <- st_transform(prop_2017_sf, st_crs(nyc_tracts_merged))

# Spatial join: properties + tracts (with ZIP)
prop_with_zip_sf <- st_join(prop_2017_sf, nyc_tracts_merged["ZIP"])

prop_with_zip <- as.data.frame(st_drop_geometry(prop_with_zip_sf))

# Sanity: how many properties have ZIP?
sum(!is.na(prop_with_zip$ZIP))

## [1] 23911

sum(is.na(prop_with_zip$ZIP))

## [1] 2643

# Drop ones without ZIP
prop_with_zip <- prop_with_zip[!is.na(prop_with_zip$ZIP), ]

# Basic summary
summary(prop_with_zip$price)

##      Min.   1st Qu.   Median     Mean  3rd Qu.    Max.
##      20    435000   600000   774501   870000  43500000

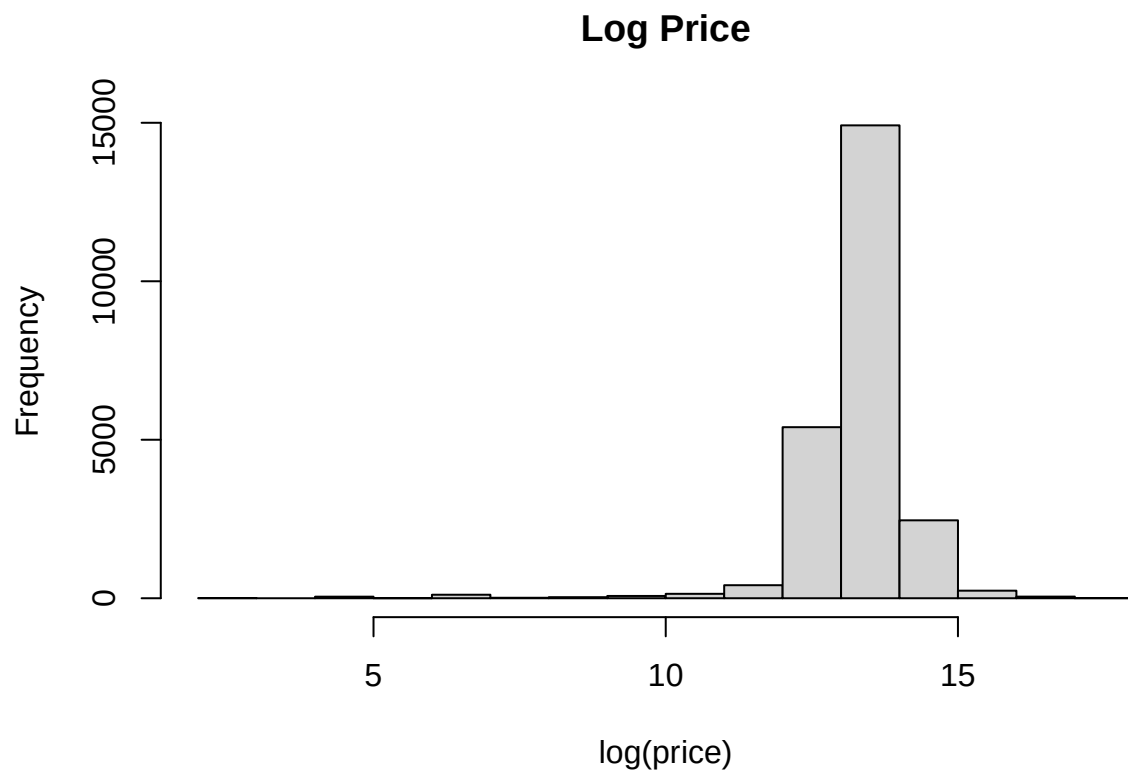
# Merge crime metrics onto property sales by ZIP
model_zip_df <- merge(
  prop_with_zip,
  crime_zip,
  by = "ZIP",
  all.x = TRUE
)

# NAs for crime + 0 (no recorded arrests)
model_zip_df$total_arrests_zip[is.na(model_zip_df$total_arrests_zip)] <- 0
model_zip_df$felony_zip[is.na(model_zip_df$felony_zip)] <- 0
model_zip_df$misdemeanor_zip[is.na(model_zip_df$misdemeanor_zip)] <- 0
model_zip_df$violation_zip[is.na(model_zip_df$violation_zip)] <- 0

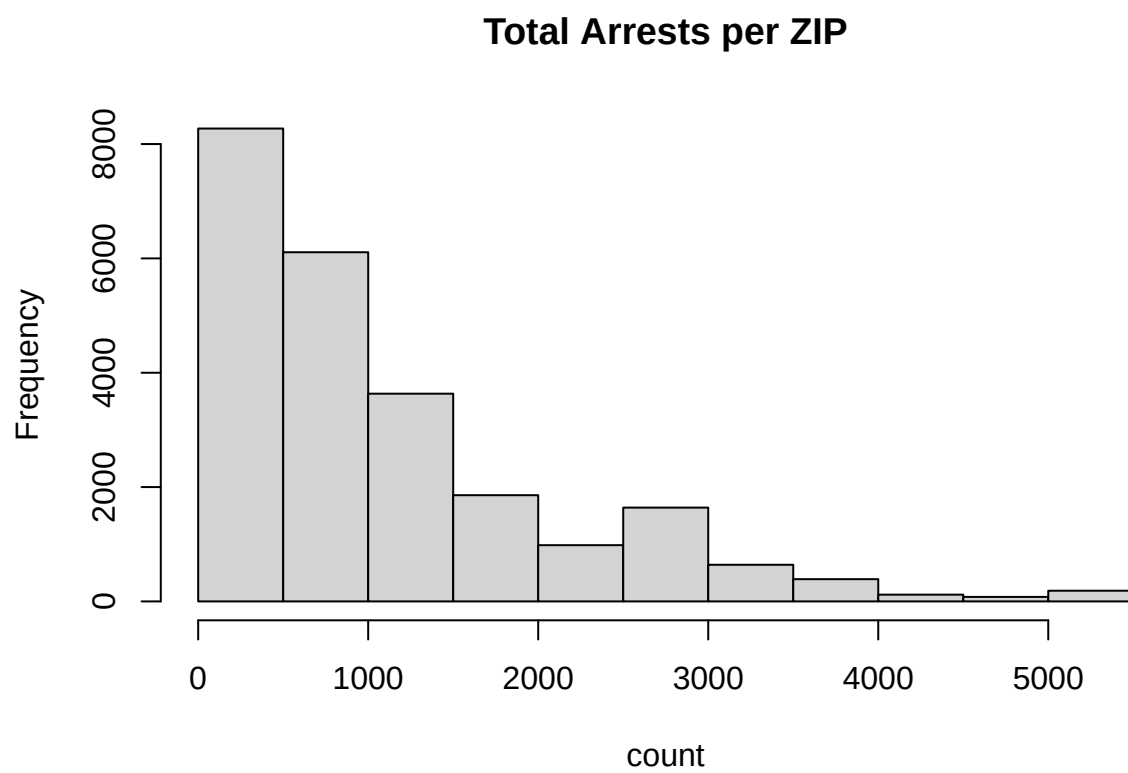
# Create log price and log sqft
model_zip_df$log_price <- log(model_zip_df$price)
model_zip_df$log_tot_sqft <- log(model_zip_df$tot_sqft + 1)

# Simple sanity plots
hist(model_zip_df$log_price, main = "Log Price", xlab = "log(price)")

```

```
hist(model_zip_df$total_arrests_zip, main = "Total Arrests per ZIP", xlab = "count")
```



```
set.seed(160)  
n <- nrow(model_zip_df)
```

```

idx <- sample(1:n, n)
train_n <- floor(0.7 * n)
valid_n <- floor(0.15 * n)

train_idx <- idx[1:train_n]
valid_idx <- idx[(train_n + 1):(train_n + valid_n)]
test_idx <- idx[(train_n + valid_n + 1):n]

train <- model_zip_df[train_idx, ]
valid <- model_zip_df[valid_idx, ]
test <- model_zip_df[test_idx, ]

nrow(train); nrow(valid); nrow(test)

## [1] 16737
## [1] 3586
## [1] 3588

rmse <- function(y, p) {
  sqrt(mean((y - p)^2, na.rm = TRUE))
}

mA <- lm(
  log_price ~ log_tot_sqft + yr_built + land_sqft + bldg_num_ctgy,
  data = train
)

summary(mA)

##
## Call:
## lm(formula = log_price ~ log_tot_sqft + yr_built + land_sqft +
##     bldg_num_ctgy, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.1781  -0.2355   0.0596   0.4139   4.0777
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.325e+01  1.532e-01  86.487  <2e-16 ***
## log_tot_sqft  1.359e-01  9.627e-03  14.121  <2e-16 ***
## yr_built     -6.927e-04  8.211e-05  -8.435  <2e-16 ***
## land_sqft     4.047e-05  3.617e-06  11.187  <2e-16 ***
## bldg_num_ctgy 1.380e-01  1.171e-02  11.783  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9313 on 16732 degrees of freedom
## Multiple R-squared:  0.03611,    Adjusted R-squared:  0.03588
## F-statistic: 156.7 on 4 and 16732 DF,  p-value: < 2.2e-16

mB <- lm(
  log_price ~ log_tot_sqft + yr_built + land_sqft + bldg_num_ctgy +

```

```

    factor(ZIP),
    data = train
)

summary(mB)$adj.r.squared

## [1] 0.246352

mC <- lm(
  log_price ~ log_tot_sqft + yr_built + land_sqft + bldg_num_ctgy +
    total_arrests_zip,
  data = train
)

summary(mC)

##
## Call:
## lm(formula = log_price ~ log_tot_sqft + yr_built + land_sqft +
##     bldg_num_ctgy + total_arrests_zip, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.0524  -0.2291   0.0579   0.3949   4.2058
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.346e+01  1.525e-01  88.318  <2e-16 ***
## log_tot_sqft    1.472e-01  9.569e-03  15.383  <2e-16 ***
## yr_built       -7.937e-04  8.164e-05  -9.722  <2e-16 ***
## land_sqft       3.243e-05  3.618e-06   8.963  <2e-16 ***
## bldg_num_ctgy    1.800e-01  1.187e-02  15.156  <2e-16 ***
## total_arrests_zip -1.279e-04  7.549e-06 -16.941  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9235 on 16731 degrees of freedom
## Multiple R-squared:  0.05237,    Adjusted R-squared:  0.05209
## F-statistic: 184.9 on 5 and 16731 DF,  p-value: < 2.2e-16

# Baseline
predA_valid <- predict(mA, newdata = valid)
predA_test  <- predict(mA, newdata = test)

rmseA_valid <- rmse(valid$log_price, predA_valid)
rmseA_test  <- rmse(test$log_price, predA_test)

# Baseline + crime (no ZIP FE)
predC_valid <- predict(mC, newdata = valid)
predC_test  <- predict(mC, newdata = test)

rmseC_valid <- rmse(valid$log_price, predC_valid)
rmseC_test  <- rmse(test$log_price, predC_test)

rmseA_valid; rmseC_valid

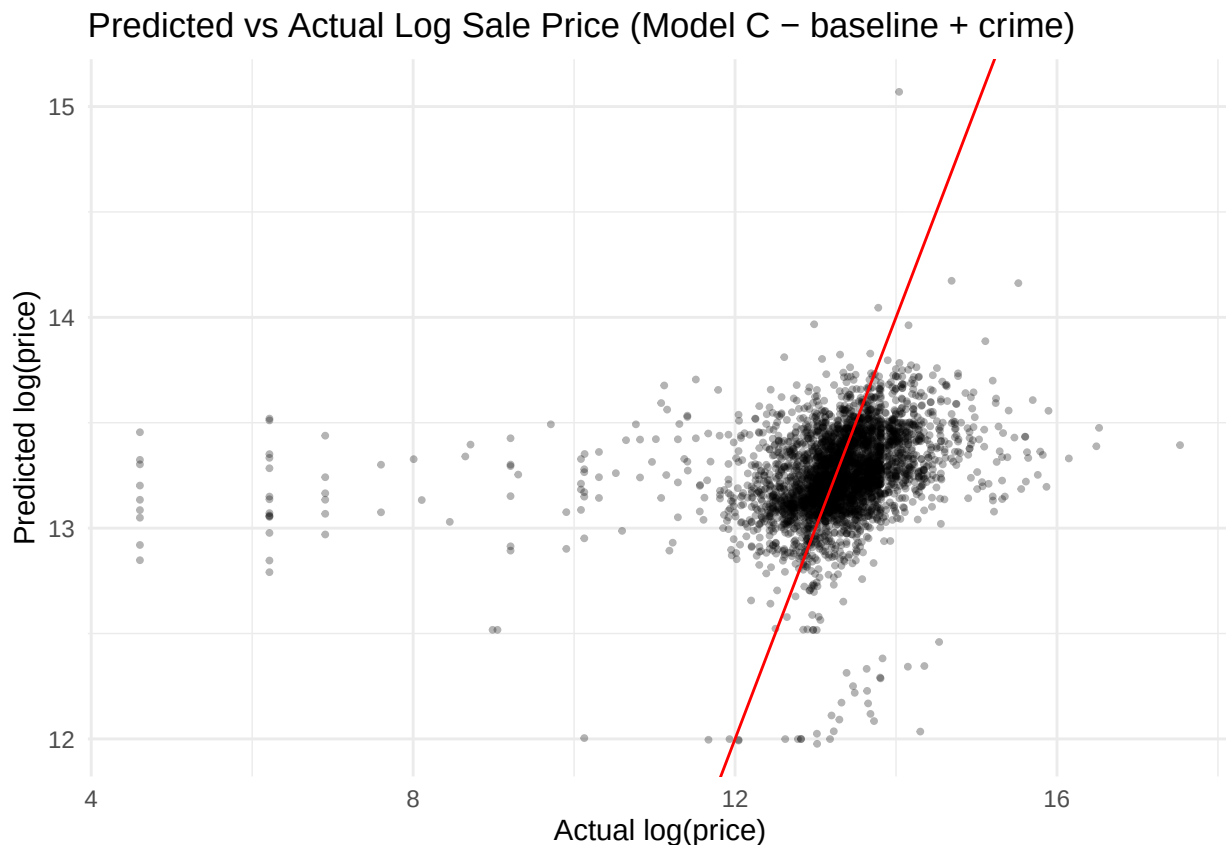
```

```
## [1] 0.9630998
## [1] 0.9520886
rmseA_test; rmseC_test

## [1] 0.9627326
## [1] 0.9525803

predC_test <- predict(mC, newdata = test)

ggplot(data.frame(
  actual = test$log_price,
  predicted = predC_test
), aes(x = actual, y = predicted)) +
  geom_point(alpha = 0.3, size = 0.7) +
  geom_abline(slope = 1, intercept = 0, color = "red") +
  theme_minimal() +
  labs(
    title = "Predicted vs Actual Log Sale Price (Model C - baseline + crime)",
    x = "Actual log(price)",
    y = "Predicted log(price)"
  )
)
```



```
library(randomForest)

## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
```

```

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##      margin

# Prepare RF training dataframe
rf_train <- train[, c("log_price", "log_tot_sqft", "yr_built",
                     "land_sqft", "bldg_num_ctgy", "total_arrests_zip")]

rf_valid <- valid[, c("log_price", "log_tot_sqft", "yr_built",
                     "land_sqft", "bldg_num_ctgy", "total_arrests_zip")]

rf_test <- test[, c("log_price", "log_tot_sqft", "yr_built",
                    "land_sqft", "bldg_num_ctgy", "total_arrests_zip")]

# Random Forest Model A (this is the baseline)
set.seed(160)

rfA <- randomForest(
  log_price ~ log_tot_sqft + yr_built + land_sqft + bldg_num_ctgy,
  data = rf_train,
  ntree = 500,
  mtry = 3,
  importance = TRUE
)

# Validation + Test predictions
rfA_pred_valid <- predict(rfA, newdata = rf_valid)
rfA_pred_test  <- predict(rfA, newdata = rf_test)

# RMSEs
rfA_valid_rmse <- rmse(rf_valid$log_price, rfA_pred_valid)
rfA_test_rmse  <- rmse(rf_test$log_price, rfA_pred_test)

rfA_valid_rmse

## [1] 0.8954844
rfA_test_rmse

## [1] 0.9284153

# Random Forest Model C (baseline + crime)
set.seed(160)

rfC <- randomForest(
  log_price ~ log_tot_sqft + yr_built + land_sqft + bldg_num_ctgy +
    total_arrests_zip,
  data = rf_train,
  ntree = 500,
  mtry = 3,
  importance = TRUE
)

rfC_pred_valid <- predict(rfC, newdata = rf_valid)

```

```

rfC_pred_test <- predict(rfC, newdata = rf_test)

rfC_valid_rmse <- rmse(rf_valid$log_price, rfC_pred_valid)
rfC_test_rmse <- rmse(rf_test$log_price, rfC_pred_test)

rfC_valid_rmse

## [1] 0.8227054

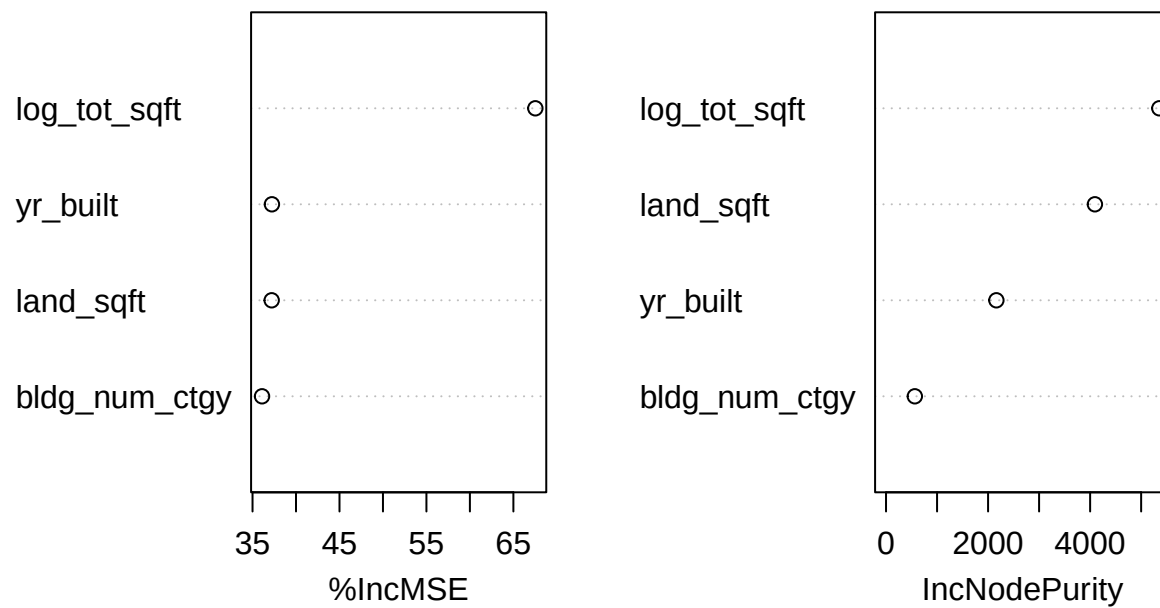
rfC_test_rmse

## [1] 0.8699469

# Variable importance plots
varImpPlot(rfA, main = "RF Baseline - Variable Importance")

```

RF Baseline – Variable Importance



```

varImpPlot(rfC, main = "RF Crime Model - Variable Importance")

```

RF Crime Model – Variable Importance

